

Elliot Uvero

Student #: 2611467

EE 446 Lab 5

Question 1: *In the notebook, the audio waveform is transformed into time-frequency features before being passed to the keyword spotting model. What is the purpose of converting raw audio into spectrogram/MFCC-style features instead of directly using the raw time-domain waveform?*

The overall objective is to transform audio data into something that can be used by a Convolutional Neural Network for the KWS model. As we have seen previously, CNN's are ideal for image classification applications and we want to find a new way to visualize sound. Analyzing raw time-domain waveform data by itself would miss specific pitches or frequencies and their amplitudes. An FFT of the raw audio data extracts these frequency components and their amplitudes which helps us identify distinct sounds and gives us additional features to work with. However, we lose temporal data with the FFT. A spectrogram consisting of FFT's taken over a time window gives us a visual heat map showing how volumes of different pitches evolve over time. We now have a picture where a time and frequency coordinate has an amplitude intensity assigned to it similar to how pixels are defined in images for each channel. This spectrogram then needs to have its frequency axis rescaled into a Mel-spectrogram which stretches low frequencies within human hearing range and squashes higher frequencies outside of human hearing range. But this Mel-Spectrogram is still too noisy and correlated. We finally apply a discrete cosine transform to get the Mel Frequency Cepstral Coefficients spectrogram which helps us de-correlate the Mel frequency bands and compress the model so it can run on a TinyML device.

Question 2: *During full integer quantization, the notebook uses a representative dataset. What is the purpose of the representative dataset, and why is it important for creating an INT8 TensorFlow Lite model?*

A representative dataset gives us something to determine the range of activation values. This calibration dataset is needed for accurate full integer quantization.

Question 3: *What does `audio_processor.get_data()` do? Refer to `input_data.py`*

In the notebook, we first initialize the `audio_processor` class object by first downloading and extracting the `speech_commands` dataset. The class object then prepares a data index which is a list of the samples organized by set and label and also processes any background noise data incorporated in the dataset. It then finishes initializing by creating a TensorFlow graph that applies any transformations to the input data. This graph loads a WAV file, decodes it, scales the volume, applies time shifts, adds in background noise, calculates a spectrogram and builds an MFCC fingerprint. When we call `audio_processor.get_data()`, it retrieves a list of the MFCC transformed sample data and a list of label indexes.

Question 4: *In the final model export step, the TensorFlow Lite model is converted into a C/C++ byte array. What are `g_model` and `g_model_len`, and why do they need to be copied correctly into the Arduino source file?*

`g_model` is a list of 8-bit values representing the actual byte code of the TensorFlow Lite KWS model and `g_model_len` is the number of bytes listed in `g_model`. `g_model` needs to be copied correctly so that the Arduino board can load the actual KWS model we trained and quantized into the board's memory. `g_model_len` also needs to be correct so that the Arduino TensorFlow Lite Verifier passes when compiling and deploying the application.

Question 5: Serial Monitor Screenshot

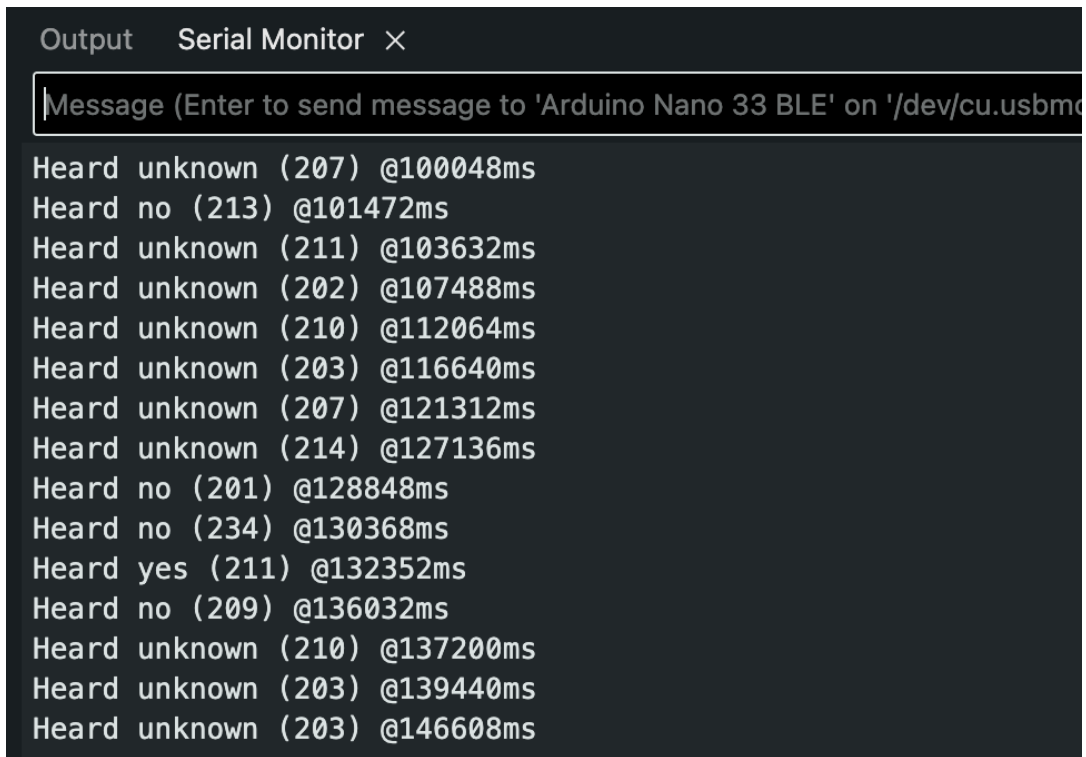


Figure 1: Arduino IDE Serial Monitor